Module 6 — Python for data work: files, APIs, pandas and testing

Learning outcomes

Build a small, reliable data-processing script that reads files or APIs, validates records, transforms data, writes outputs and can be tested.

1. Files and formats

CSV is tabular text; JSON represents nested structured data; Parquet is a columnar analytical format. Choose formats based on consumers, size and schema needs. Use explicit encoding and date/number parsing.

```
import csv
with open("orders.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(row["order_id"])
```

2. APIs

Know HTTP methods, status codes, headers, authentication, pagination, rate limits, timeouts and JSON parsing. Use timeouts. Retry transient errors (429/5xx) with bounded exponential backoff and jitter; do not endlessly retry permanent 4xx errors.

3. pandas essentials

• Inspect: `head`, `sample`, `shape`, `info`, `describe`, `dtypes`.

• Missingness: `isna`, `notna`, `fillna`, `dropna`.

• Cleaning: `astype`, `to_numeric`, `to_datetime`, `drop_duplicates`.

• Transform: `assign`, vectorised operations, `map`, `groupby`, `agg`.

• Combine: `merge`, `concat`.

• Reshape: `pivot_table`, `melt`.

• Export: `to_csv`, `to_parquet`, `to_sql`.

4. Merge validation

```
joined = orders.merge(
    customers,
    on="customer_id",
    how="left",
    validate="many_to_one"
)
```

Use `validate` to catch unexpected key cardinality. Check unmatched keys after merge.

5. Missing values

Do not automatically fill every missing numeric value with zero. Ask whether missing means unknown, not applicable or true zero. Report missingness before treatment. For a date column, coercion may produce `NaT`; count and inspect those rows.

6. Testing and quality

Use unit tests for deterministic transformations; integration tests for database/API boundaries; data tests for uniqueness, not-null, accepted values, relationships, freshness and volume. Keep raw inputs and rejected records with reasons.

Guided exercises and solutions

Assume `orders.csv` has `order_id, customer_id, order_ts, status, amount`.

1. Parse dates and numeric amounts.

```
import pandas as pd
df = pd.read_csv("orders.csv")
df["order_ts"] = pd.to_datetime(df["order_ts"], errors="coerce", utc=True)
df["amount"] = pd.to_numeric(df["amount"], errors="coerce")
```

2. Report invalid required fields.

```
invalid = df[
    df["order_id"].isna()
```

```
    | df["customer_id"].isna()
    | df["order_ts"].isna()
    | df["amount"].isna()
]
```

3. Create monthly completed revenue.

```
valid = df[df["status"].eq("COMPLETED")].copy()
valid["month"] = valid["order_ts"].dt.to_period("M")
monthly = valid.groupby("month", as_index=False)["amount"].sum()
```

4. Detect duplicate order IDs and retain latest update.

```
df = df.sort_values(["updated_at", "ingested_at"])
deduped = df.drop_duplicates("order_id", keep="last")
```

This requires parsed timestamps and a defined tie rule.

5. Merge customer city, requiring one customer row per customer ID.

```
result = orders.merge(
    customers, on="customer_id", how="left", validate="many_to_one",
    indicator=True
)
unmatched = result[result["_merge"] == "left_only"]
```

Independent exercises

6. Calculate category revenue and item count from order lines.

7. Compare row count before and after a merge; explain unexpected growth.

8. Write a function that returns accepted and rejected rows with reason.

9. Read a 5 GB CSV in chunks and aggregate by month.

10. Write a unit test for a function that standardises status values.

11. Describe how to make a daily API ingestion job safe to rerun.

Solution patterns

• 6: `groupby("category").agg(revenue=("line_value","sum"), lines=("order_item_id","count"))`.

• 7: inspect key uniqueness, join cardinality, duplicate keys and null join keys; use pandas `validate`.

• 8: separate validation rules from persistence; preserve original row and error list.

• 9: `read_csv(..., chunksize=...)`, aggregate partial results, merge partial aggregates, checkpoint output.

• 10: test known mappings, unknown status, whitespace/case and null.

• 11: stable business key, deterministic transformation, upsert, run metadata and safe checkpoint.

Mini project

Create `daily_order_quality.py`:

1. Read CSV.

2. Validate required fields.

3. Parse timestamp and amount.

4. Reject invalid rows with reasons.

5. Deduplicate by order ID and latest update.

6. Produce daily revenue and quality summary.

7. Write output CSVs and run metadata.

8. Add tests for empty input, invalid amount, duplicate ID and missing timestamp.

External practice for this module

1. Kaggle Learn — Python — complete the Python micro-course if core syntax is still weak.

2. Kaggle Learn — Pandas — complete all lessons and exercises on indexing, grouping, missing data, merging and data types.

3. LeetCode — 30 Days of Pandas — complete the study plan; write down the pandas operation used and its SQL equivalent.

4. StrataScratch — select Python/Pandas interview problems involving groupby, joins, ranking and data cleaning.

5. pytest documentation — getting started — write tests for the mini-project in this module.

Exit requirement: complete 15 Pandas exercises and build one script that reads a file, validates, transforms, rejects bad rows with reasons, and writes reproducible outputs. Test empty input, duplicates and invalid types.